UPI Fraud Detection: Project Plan

Oct 24, 2026 · @esmoli

Verdict

A solid idea. It solves a real problem in India, it is demo-friendly, and it covers the full stack (ML, backend, mobile client, payments), which is much stronger than another notebook project. A few things need adjusting to make it realistic.

Idea rating: 7.5/10. **Execution potential:** 9/10, if you lead with rules, reputation, and explanations, and keep ML as a supporting layer.

Reality checks

1. **Training data is the biggest problem.** Real UPI transaction data belongs to NPCI and banks, so you can't get it. Public datasets (like PaySim or Kaggle "UPI fraud" sets) are mostly synthetic. A model trained only on those will look great in a notebook but won't prove much in practice. Treat ML as one layer, not the whole product.
2. **You can't intercept scans in GPay or PhonePe.** Your app has to be its own scanner: the user scans the QR in your app, you analyze it, and then you hand off to a UPI intent or your gateway to pay.
3. **"Authentic or not" should be a risk score, not a verdict.** No system can say definitively that a UPI ID is legitimate. "Low / Medium / High risk, and here's why" is more honest and more useful.
4. **Detecting fraud on incoming money is harder.** You only see the payer's details if you're a merchant using a gateway webhook, or if the user pastes in an SMS or notification. Design it around those two cases.

What a UPI QR actually contains

A QR encodes a link like `upi://pay?pa=name@bank&pn=Name&am=500&tn=note&mc=5411`. That gives you the payee VPA, name, amount, merchant category code, and note. This is your feature source.

Signals (hybrid of rules and ML)

Rule-based

- Handle isn't a valid PSP handle.
- Payee name in the QR doesn't match the name returned by VPA validation. Gateways like Razorpay and Cashfree offer VPA validation APIs; this is probably your strongest check.
- A "pay to receive money" request. A QR is only ever for paying, so this is always a scam, and it makes a strong demo moment.
- Collect requests disguised as payments.
- Lookalike merchant names, such as a typo of a known brand.
- Merchant category code doesn't match the claimed business.

Crowd-sourced reputation

Users report fraudulent VPAs, and the report count and recency feed a score. This is your moat, since it builds data nobody else has.

Graph analysis

One VPA receiving small amounts from many unrelated accounts is a mule-account pattern.

ML layer

Anomaly detection or gradient boosting (XGBoost) on amount, time, velocity, and new-payee features, trained on synthetic data plus your own collected reports.

Physical tampering

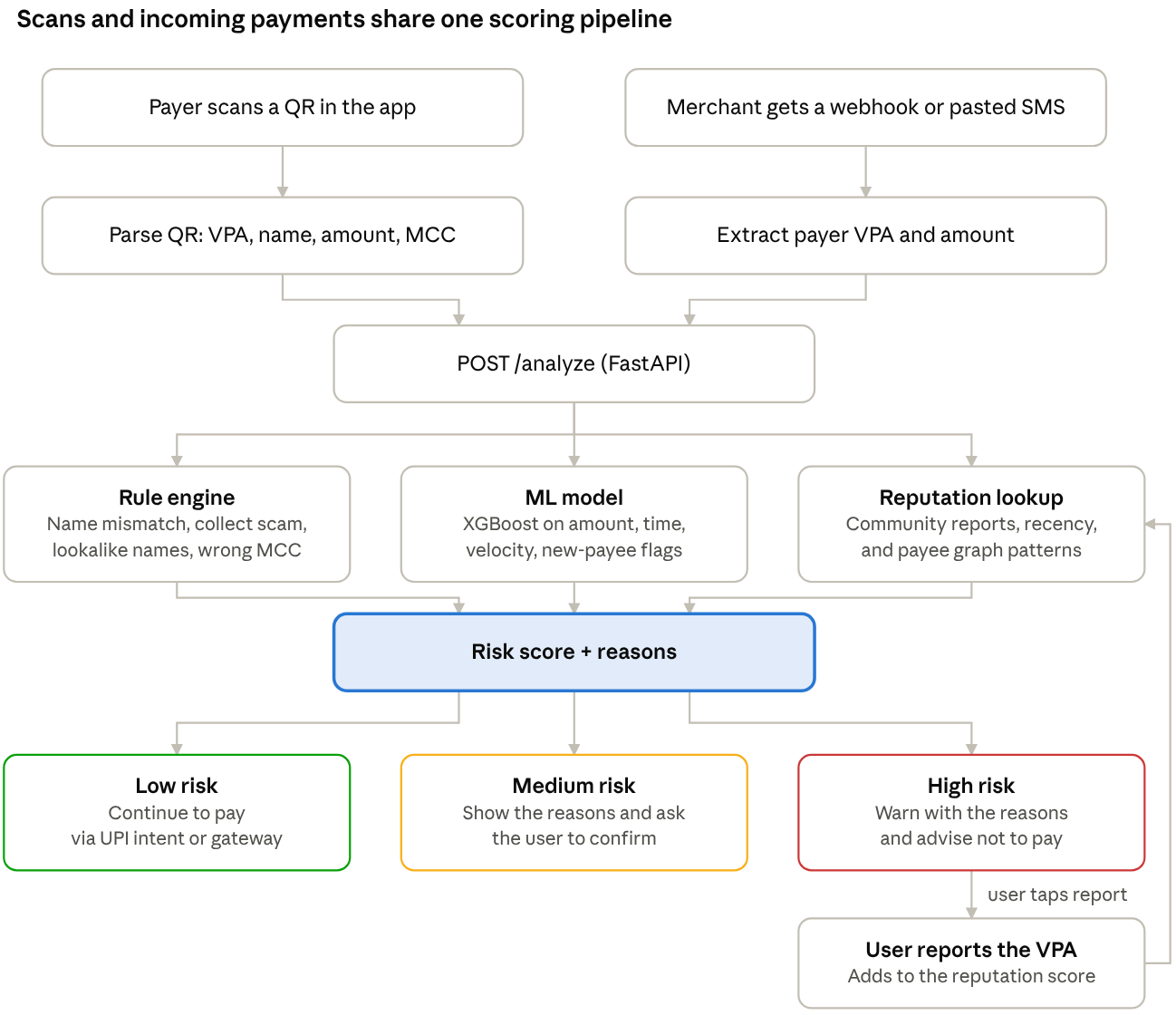
Warn users when a shop QR looks like a sticker pasted over another one. This is a common real-world scam.

Suggested architecture

- **Client:** Flutter or React Native, with a QR scanner, risk screen, and report button.
- **Server:** FastAPI, with an `/analyze` endpoint that parses the QR, runs the rules, then the ML model, then the reputation lookup, and returns a score with reasons.

- **Database:** PostgreSQL for reports, Redis for fast VPA lookups, and a graph store for relationship analysis.
- **Payments:** Razorpay or Cashfree in test/sandbox mode for the demo.
- **Receiving side:** a webhook that scores each incoming payer VPA and alerts the merchant.

Workflow



runtime workflow · 2 entry points, 3 checks, 3 outcomes

Scans and incoming payments share one /analyze call; user reports feed back into the reputation check.

How to stand out

Existing apps already show verified merchant names, and NPCI and banks run their own fraud systems. So don't pitch "fraud detection"; pitch **explainability and community intelligence**: "This QR is risky because the name doesn't match the registered account, it was reported 14 times this week, and it asks you to pay to receive money." People trust reasons more than a bare score.

MVP in about 4 weeks

1. QR parser and rule engine.
2. Mobile scanner with a risk screen.
3. VPA validation and name match via gateway sandbox.
4. Reporting system and reputation score.
5. ML model and merchant webhook last.

Next steps

- Risk-scoring logic (weights, thresholds, reason generation)
- API design for `/analyze`, `/report`, and the webhook
- Database schema for reports, VPAs, and reputation